

q-wallet

Quantum-Safe Sui

Post-quantum keys on Sui. We build Q-Wallet: one phrase, two keys, and an account that moves to ML-DSA-65 without changing its address.

day1-quantum-safe-wallet.vercel.app

[KEYS, DOLLARS AND BITCOIN · DAY 1 · SUIHUB LAGOS]

RURU (@MINTING_RURU)

[THE ONE IDEA]

Quantum breaks keys, not addresses.

So change the key, keep the address.

Ed25519, secp256k1 and secp256r1 all fall to Shor's algorithm on a large enough quantum computer. Every signature you send publishes your public key, so accounts that have signed are already exposed. Sui adds a post-quantum key, ML-DSA-65, and aliases that let an account switch to it.

[ML-DSA-65]

ML-DSA-65: the key Sui added

NIST FIPS 204, built on lattices. Sizes measured on devnet with @mysten/sui 2.39.0.

	Ed25519 (today)	ML-DSA-65 (post-quantum)
Standard	RFC 8032	NIST FIPS 204
Hard problem	Elliptic curve discrete log	Module lattices
Sui signature flag	0x00	0x07
Public key	32 bytes	1,952 bytes
Signature	64 bytes	3,309 bytes
In the transaction (flag + signature + key)	97 bytes	5,262 bytes
Signing	Deterministic	Hedged: fresh randomness each time

[WHERE IT IS LIVE]

Live status: devnet only, for now

Feature flag mldsa65_auth, read from each network's GraphQL protocolConfigs on 8 Nov 2026.

DEVNET	TESTNET	MAINNET
protocol 141	protocol 140	protocol 141
On	**Off**	**Off**
mldsa65_auth = true. Q-Wallet runs here.	Planned for the end of 2026.	Upgraded, flag still off. Targeted for Q1 2027.

TYPESCRIPT

```
// src/lib/sui.ts, readPqFlags(): one request per network
body: JSON.stringify({ query: '{ protocolConfigs { protocolVersion featureFlags { key value } } }' })
const f = c.featureFlags.find((x) ⇒ x.key ═══ 'mldsa65_auth');
// a failed read returns mldsa: null (unknown), never false (off)
```

01

One phrase, two keys

[SRC/LIB/KEYS.TS]

Derive both keys from one phrase

Abridged from the repo. Different paths and schemes, so the same 12 words give two addresses.

TYPESCRIPT

```typescript
import { Ed25519Keypair } from '@mysten/sui/keypairs/ed25519';
import { MLDSA65Keypair } from '@mysten/sui/keypairs/mldsa65';
import { generateMnemonic, validateMnemonic } from '@scure/bip39';

export function newPhrase() { return generateMnemonic(wordlist, 128); }   // 12 words

export function keysFrom(saved: Saved): Keys {
  if (!validateMnemonic(mnemonic, wordlist)) throw new Error('That is not a valid 12 or 24 word phrase');
  return {
    classic: Ed25519Keypair.deriveKeypair(mnemonic),   // m/44'/784'/0'/0'/0'   address X
    quantum: MLDSA65Keypair.deriveKeypair(mnemonic),   // m/94'/784'/0'/0'/0'   address Q
  };
}
```

02

Move the account, keep the address

[THE MIGRATION]

Five transactions, one address

X is the classic address, Q the quantum-safe one. X never changes; only the key allowed to sign for it does.

01

Enable

enable, signed by the classic key. X now has an alias list: [X].

02

Add Q

add(Q), signed by the classic key again, in its own transaction. List: [X, Q].

03

Send as X

Sender X, signature from Q. Validators accept it because Q is on the list.

04

Retire X

remove(X), signed by Q. List: [Q]. Only after step 3 has worked.

05

Refused

The classic key is now rejected: Required Signature from Q is absent.

[SRC/LIB/SUI.TS]

Aliases: three framework calls

No custom Move. The AddressAliases id is derived from 0xa and the owner, so no indexer either.

TYPESCRIPT

```typescript
// sui::address_alias functions are entry: each must be its own top-level PTB command.
export const aliasEnable = (tx: Transaction) =>
  tx.moveCall({ target: '0x2::address_alias::enable', arguments: [tx.object(ALIAS_STATE_ID)] });
export const aliasAdd = (tx: Transaction, aliasesId: string, alias: string) =>
  tx.moveCall({ target: '0x2::address_alias::add', arguments: [tx.object(aliasesId), tx.pure.address(alias)] });
export const aliasRemove = (tx: Transaction, aliasesId: string, alias: string) =>
  tx.moveCall({ target: '0x2::address_alias::remove', arguments: [tx.object(aliasesId), tx.pure.address(alias)] });

deriveObjectID(ALIAS_STATE_ID, '0x2::address_alias::AliasKey', bcs.Address.serialize(owner).toBytes());

// App.tsx, step 4: sender X, signed by the quantum-safe key
await run('Retire classic key', X, keys.quantum, (tx) => aliasRemove(tx, aliases.objectId, X));
```

[VERIFIED ON DEVNET]

Proof: the run on the live site

pnpm e2e against the Vercel app, 6 Nov 2026, 10:58 UTC. X = 0x4208…b52a, Q = 0xeb13…9a98.

Step	Sender	Signed by	Digest
Fund Q from X	X	Ed25519	`Cb2EUvqYo1kntqiv3mz9RHFu67vD4mdQwsYVBHYoTjF4`
Send with ML-DSA-65	Q	ML-DSA-65	`FFAHNrfe18HAGMBvhJWuQdZTQjuSWrB2E3Fr2FQSufDz`
Turn on aliases	X	Ed25519	`4ihGXT3RM3tDeQyu8od6kV1QgtC1ZSeCMXRvXPU51VYm`
Add Q as an alias	X	Ed25519	`5YHE147QArQ3gyYGniWSsiBstXae5yRvqdLBtBXEDttR`
Send as X	X	ML-DSA-65	`86bwyUpu6yUhoqcruHEhZsGyNu53CofXvEUmfWzWF2uK`
Retire the classic key	X	ML-DSA-65	`3jwSMrHo3t7ngnCaKVpT4jcSmrvRVanSTPBH859Eo5Jx`
Classic key afterwards	X	Ed25519	`Refused: Required Signature from 0xeb13...9a98 is absent`

[GOTCHAS]

Six things that will bite you

All of them came up while building and running Q-Wallet.

01 Devnet only

mldsa65_auth is off on testnet and mainnet. Check the flag before pointing the app anywhere else.

02 5 KB per signature

5,262 bytes against 97. Fine for one user; count it before batching many signers into one flow.

03 Random signatures

Same bytes, a new valid signature each time. Never use a signature as an id or a cache key.

04 Order matters

enable and add go in separate transactions. Remove the classic key only after Q has signed as X.

05 Faucet limits

The devnet faucet rate-limits by IP. In a full room, fund one classic account per pair.

06 Keys in localStorage

Fine for a devnet class, never for real money. Clearing site data loses the phrase.

[BUILD IT]

Run Q-Wallet yourself

Node 22+, pnpm and a browser. No environment variables. The only server code is a devnet faucet relay.

SHELL

```
git clone https://github.com/Nuel-osas/day1-quantum-safe-wallet
cd day1-quantum-safe-wallet
pnpm install
pnpm dev      # http://localhost:5292
pnpm build    # typecheck, then production build
pnpm e2e      # clicks the whole flow on devnet

# the same run against the live site
APP_URL=https://day1-quantum-safe-wallet.vercel.app pnpm e2e
```

01 **Create a new wallet**
Two addresses appear: X and Q

02 **Fund**
Faucet X, then fund Q from X

03 **Send with ML-DSA-65**
Watch the size bar: 5,262 bytes

04 **Upgrade X**
Enable, add Q, send as X, retire X

05 **Prove it**
The classic key is refused

[TAKE HOME]

Change the key, keep the address.

Works on devnet today.
Not on testnet or mainnet yet.

ML-DSA-65 is a real Sui key type, derived from the phrase users already have. Aliases move an existing account onto it with three framework calls. Prepare now: derive the key, rehearse the migration, budget 5 KB per signature. Ship it to users once the flag is on and their wallet supports it.